

Universidad Autónoma Tomás Frías
Carrera de Ingeniería Informática

INF781 — Seguridad de Software

TAREA PRÁCTICA

**Aplicación Intencionalmente Insegura en PHP Puro:
Construir para Comprender las Vulnerabilidades**

Docente: **M. Sc. Huáscar Fedor Gonzales Guzmán**

Modalidad: **Individual**

Tecnología: **PHP 8.3+ puro (sin frameworks) + PostgreSQL**

Plazo: **1 semana desde la fecha de asignación**

Entregable: **Código en GitHub + Informe PDF**

Gestión 2025 — Potosí, Bolivia

1. Introducción y Objetivo

En las Guías 1 a 4 de esta materia has implementado múltiples capas de seguridad en una aplicación Laravel: configuración segura de entorno, validación y sanitización de entradas, hashing robusto de contraseñas, manejo seguro de sesiones y protección CSRF. Pero, ¿qué pasaría si no existiera ninguna de estas protecciones?

El objetivo de esta tarea es que construyas una aplicación en PHP puro (sin ningún framework) que sea intencionalmente insegura —una que omita deliberadamente todas las buenas prácticas aprendidas— para que puedas demostrar, atacar y documentar cada vulnerabilidad. Al programar sin framework, comprenderás en profundidad todo lo que Laravel hace por ti automáticamente y por qué cada capa de seguridad es indispensable.

Advertencia Ética

Esta aplicación debe ejecutarse únicamente en entorno local (localhost). Nunca la despliegues en un servidor público. El propósito es exclusivamente educativo: entender las vulnerabilidades para poder prevenirlas.

2. Descripción de la Aplicación

Construirás una aplicación web llamada "InsecureNotes", un sistema simple de notas personales con registro, login y CRUD de notas. La aplicación debe desarrollarse con PHP 8.3+ puro (sin Laravel, Symfony ni ningún framework) y PostgreSQL como base de datos.

2.1. Funcionalidades requeridas

1. Registro de usuario (nombre, email, contraseña).
2. Login y logout con manejo manual de sesiones (`session_start()`).
3. Crear, listar, editar y eliminar notas personales (título y contenido).
4. Página de perfil donde se muestre el nombre del usuario.

2.2. Requisitos técnicos

- Servidor de desarrollo: `php -S localhost:8000` (servidor built-in de PHP).
- Conexión a PostgreSQL mediante `pg_connect()` o PDO sin prepared statements.
- Sin Composer, sin autoloaders, sin librerías externas.
- Archivos `.php` directos con HTML embebido (sin motor de plantillas).

¿Por qué PHP puro?

Laravel incluye decenas de protecciones automáticas (escape en Blade, CSRF middleware, bcrypt en Hash::make, Form Requests). Al construir con PHP puro, no hay ningún «parachoques»: cada vulnerabilidad será evidente y cada protección deberá implementarse manualmente. Esto te permite entender qué resuelve cada mecanismo de seguridad a nivel fundamental.

3. Vulnerabilidades a Implementar

Debes introducir intencionalmente las siguientes vulnerabilidades, cada una correspondiente a lo que aprendiste a prevenir en las guías:

1

Exposición de Información Sensible (Guía 1)

Relación con Guía 1: La Guía 1 enseñó a ocultar detalles internos con el Handler de excepciones, usar APP_DEBUG=false en producción y proteger credenciales con variables de entorno (.env).

Lo que debes hacer en PHP puro:

- Activar la visualización de errores directamente en el código:

```
<?php
// config.php
ini_set('display_errors', 1);
ini_set('display_startup_errors', 1);
error_reporting(E_ALL);

// Credenciales hardcodeadas directamente en el código
$db_host = 'localhost';
$db_name = 'insecure_notes';
$db_user = 'postgres';
$db_pass = 'mi_password_secreto';

$conn = pg_connect("host=$db_host dbname=$db_name
                    user=$db_user password=$db_pass");
```

- No atrapar excepciones ni errores de base de datos: dejar que PHP muestre los errores completos con rutas de archivos, líneas de código y la cadena de conexión.

Ataque a demostrar: (CAPTURA DE PANTALLA 1)

Provocar un error (por ejemplo, acceder a una tabla inexistente o enviar una consulta SQL malformada) y capturar la pantalla donde PHP revele: ruta completa del archivo, credenciales de conexión, nombre de la base de datos y versión de PHP.

2

Sin Validación ni Sanitización — XSS e Inyección SQL (Guía 2)

Relación con Guía 2: La Guía 2 enseñó a validar entradas con Form Requests de Laravel, aplicar htmlspecialchars() y usar la directiva {{ }} de Blade para escape automático.

Lo que debes hacer en PHP puro:

- **XSS:** Imprimir datos del usuario directamente con echo sin htmlspecialchars():

```
<!-- notas/index.php -->
<?php foreach ($notas as $nota): ?>
    <h3><?php echo $nota['titulo']; ?></h3>
    <p><?php echo $nota['contenido']; ?></p>
<?php endforeach; ?>
```

- **Inyección SQL:** Concatenar la entrada del usuario directamente en las consultas SQL sin prepared statements:

```
// notas/guardar.php
$titulo = $_POST['titulo'];
$contenido = $_POST['contenido'];
$user_id = $_SESSION['user_id'];

$query = "INSERT INTO notas (titulo, contenido, user_id)
VALUES ('$titulo', '$contenido', $user_id)";
pg_query($conn, $query);
```

- No validar tipo de dato, longitud, ni formato de ningún campo.

Ataques a demostrar: (CAPTURAS DE PANTALLA 3 y 4)

A) XSS: Crear una nota con el siguiente contenido y verificar que el script se ejecuta al visualizar las notas:

```
<script>alert('XSS: Cookie = ' + document.cookie)</script>
```

B) Inyección SQL: En el campo título de una nota, ingresar el siguiente payload para extraer datos de la tabla users:

```
'); SELECT email, password FROM users; --
```

También intentar un payload de eliminación para demostrar el riesgo máximo:

```
'); DROP TABLE notas; --
```

Nota sobre Inyección SQL

Esta vulnerabilidad no existía explícitamente en la Guía 2 porque Laravel usa Eloquent con query bindings que la previene automáticamente. En PHP puro, sin prepared statements, la inyección SQL es trivial. Esto demuestra cuánto protege el ORM de Laravel sin que el desarrollador lo note.

3 Hashing Inseguro de Contraseñas (Guía 3)

Relación con Guía 3: La Guía 3 enseñó a usar Hash::make() (bcrypt/Argon2id) y la regla Password de Laravel para contraseñas fuertes.

Lo que debes hacer en PHP puro:

- Almacenar contraseñas usando MD5 directo:

```
// registro.php
$password = $_POST['password']; // Sin validar fortaleza
$hash = md5($password);         // Hash inseguro

$query = "INSERT INTO users (nombre, email, password)
        VALUES ('$nombre', '$email', '$hash')";
pg_query($conn, $query);
```

- No aplicar ninguna regla de fortaleza: aceptar contraseñas como "123" o "a".
- En el login, comparar con MD5 directo:

```
// login.php
$email = $_POST['email'];
$password = $_POST['password'];

$query = "SELECT * FROM users WHERE email = '$email'
        AND password = '" . md5($password) . "'";
$result = pg_query($conn, $query);
```

Ataque a demostrar: (CAPTURA DE PANTALLA 4)

1. Registrar un usuario con contraseña débil (ej: "password" o "123456").
2. Acceder a la base de datos con psql y consultar la tabla users para mostrar el hash MD5.
3. Usar un sitio como crackstation.net o una tabla rainbow para revertir el hash en segundos.
4. Comparar: generar el hash bcrypt del mismo password con `password_hash('password', PASSWORD_BCRYPT)` y explicar por qué este no puede revertirse con tablas rainbow.

4

Sesiones Inseguras y Sin Protección CSRF (Guía 4)

Relación con Guía 4: La Guía 4 enseñó a usar el driver database para sesiones, cookies HttpOnly/Secure/SameSite, regenerar el session ID tras login y proteger formularios con @csrf.

Lo que debes hacer en PHP puro:

- Usar sesiones con configuración insegura de cookies:

```
// config.php (antes de session_start)
ini_set('session.cookie_httponly', 0); // JS puede leer la cookie
ini_set('session.cookie_samesite', ''); // Sin restricción de origen
ini_set('session.cookie_secure', 0); // Funciona sin HTTPS
ini_set('session.use_strict_mode', 0); // Acepta IDs arbitrarios

session_start();
```

- No llamar a session_regenerate_id() después del login.
- No implementar ningún mecanismo de token CSRF en los formularios. Los formularios envían directamente sin validación de origen:

```
<!-- notas/crear.php -->
<form method="POST" action="guardar.php">
  <!-- Sin token CSRF -->
  <input name="titulo" placeholder="Título">
  <textarea name="contenido"></textarea>
  <button type="submit">Guardar</button>
</form>
```

Ataques a demostrar: (CAPTURAS DE PANTALLA 5, 6 y 7)

A) Robo de cookie por XSS: Combinando con la vulnerabilidad 2, mostrar que document.cookie es accesible desde JavaScript porque HttpOnly está desactivado. El script inyectado debe mostrar el PHPSESSID.

B) Ataque CSRF: Crear un archivo HTML separado (atacante.html) que contenga un formulario oculto que apunte al endpoint de eliminación de notas:

```
<!-- ataques/atacante.html -->
<!-- Abrir en el mismo navegador donde la víctima tiene sesión -->
<html>
<body>
  <h1>Ganaste un premio. Haz clic aquí.</h1>
  <form id="csrf-attack" method="POST"
    action="http://localhost:8000/notas/eliminar.php">
    <input type="hidden" name="id" value="1">
  </form>
  <script>
    document.getElementById("csrf-attack").submit();
  </script>
```

```
</body>
</html>
```

Al abrir atacante.html en el mismo navegador donde existe una sesión activa, la nota con id=1 debe eliminarse sin autorización del usuario.

C) Fijación de sesión: Demostrar que si un atacante envía una URL con un PHPSESSID predeterminado (ej: `http://localhost:8000/login.php?PHPSESSID=abc123`), y la víctima inicia sesión sin que el ID se regenere, el atacante puede secuestrar la sesión usando esa misma cookie.

4. Estructura del Proyecto en GitHub

Tu repositorio debe tener la siguiente estructura:

```
insecure-notes/
├── config.php           # Conexión BD + ini_set inseguros
├── index.php            # Página principal / redirección
├── registro.php         # Formulario + lógica de registro
├── login.php            # Formulario + lógica de login
├── logout.php           # Destruir sesión
├── perfil.php           # Página de perfil del usuario
├── notas/
│   ├── index.php        # Listar notas del usuario
│   ├── crear.php        # Formulario de creación
│   ├── guardar.php      # Procesar INSERT
│   ├── editar.php       # Formulario de edición
│   ├── actualizar.php   # Procesar UPDATE
│   └── eliminar.php     # Procesar DELETE
├── sql/
│   └── schema.sql       # CREATE TABLE users, notas
├── ataques/
│   └── atacante.html    # Página CSRF del atacante
└── README.md
```

4.1. Script SQL requerido

El archivo `sql/schema.sql` debe contener la creación de las tablas:

```
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    nombre VARCHAR(100),
    email VARCHAR(150),
    password VARCHAR(100) -- Almacenará MD5 (32 chars)
);

CREATE TABLE notas (
    id SERIAL PRIMARY KEY,
    titulo VARCHAR(200),
```

```
contenido TEXT,  
user_id INTEGER REFERENCES users(id),  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Sobre el README.md

El README debe explicar brevemente: 1) el propósito educativo del proyecto, 2) cómo instalar (crear BD, importar schema.sql, iniciar servidor), 3) una advertencia visible de que NUNCA debe desplegarse en producción.

5. Informe Escrito (PDF)

Además del código, debes entregar un informe en formato Word con la siguiente estructura:

5.1. Portada

Incluir: nombre de la materia, título de la tarea, nombre completo del estudiante, fecha de entrega.

5.2. Para cada vulnerabilidad (4 secciones)

Cada sección del informe debe contener:

1. **Código vulnerable:** Fragmentos del código PHP que introducen la vulnerabilidad, con explicación línea por línea de por qué es inseguro.
2. **Demostración del ataque:** Capturas de pantalla del ataque siendo ejecutado exitosamente (mínimo 2 capturas por vulnerabilidad) con explicación de la captura de pantalla.
3. **Impacto real:** ¿Qué podría lograr un atacante en un sistema real con esta vulnerabilidad? Describir consecuencias concretas.
4. **Corrección con la Guía correspondiente:** Explicar qué técnica o mecanismo de la guía de Laravel previene este ataque. Adicionalmente, mostrar cómo se corregiría en PHP puro (ej: htmlspecialchars(), password_hash(), prepared statements, tokens CSRF manuales).

5.3. Reflexión final

Un párrafo de conclusión personal (mínimo 200 palabras) respondiendo:

- ¿Qué aprendiste al construir una aplicación insegura en PHP puro que no habrías comprendido solo con implementar las protecciones en Laravel?
- ¿Cuál vulnerabilidad te pareció la más peligrosa y por qué?
- ¿Qué protecciones automáticas de Laravel valoras más después de esta experiencia?

6. Rúbrica de Evaluación

Criterio	Pts	Descripción
App funcional (CRUD + Auth)	10	La aplicación PHP pura se ejecuta correctamente: registro, login, crear, listar, editar y eliminar notas con PostgreSQL.
Vuln. 1: Exposición de info	10	display_errors activado, credenciales hardcodeadas en config.php, error de BD muestra ruta y datos sensibles. Ataque demostrado con captura.
Vuln. 2: XSS	10	echo sin htmlspecialchars(). Payload <code><script>alert(cookie)</script></code> ejecutado y documentado con capturas.
Vuln. 2b: Inyección SQL	10	Consultas con concatenación directa (sin prepared statements). Payload de extracción o destrucción demostrado con capturas.
Vuln. 3: Hashing MD5	15	Contraseña almacenada con md5(), sin reglas de fortaleza. Hash crackeado y comparación con password_hash() documentada.
Vuln. 4: Sesión/CSRF	15	Cookies sin HttpOnly, sin session_regenerate_id(), sin token CSRF. Los tres ataques (cookie + CSRF + fijación) demostrados con capturas.
Informe: código explicado	10	Fragmentos de código PHP vulnerable con explicación clara línea por línea de cada inseguridad.
Informe: impacto real	5	Descripción realista de consecuencias en un sistema en producción para cada vulnerabilidad.
Informe: tabla + reflexión	10	Tabla comparativa completa (PHP puro vs Laravel) y reflexión personal de al menos 200 palabras.

Repositorio GitHub	5	Estructura organizada, README con advertencia, schema.sql incluido, archivo atacante.html presente.
TOTAL	100	

7. Entregables

1. **Enlace al repositorio GitHub** con el proyecto InsecureNotes en PHP puro (repositorio privado, agregar al docente como colaborador).
2. **Informe en formato PDF** con la estructura descrita en la sección 5.

8. Recomendaciones Finales

- **Trabaja en local:** Usa `php -S localhost:8000` y accede solo desde el navegador local.
- **Inicia con el schema SQL:** Crea primero la base de datos y las tablas, luego construye la aplicación archivo por archivo.
- **No uses ningún framework ni librería:** El punto es experimentar PHP sin protecciones automáticas. Sin Composer, sin autoloaders.
- **Documenta mientras desarrollas:** Toma las capturas de pantalla en el momento del ataque, no intentes recrearlas después.
- **Consulta las guías:** Para la sección de corrección, relee las guías 1-4 y referencia las técnicas específicas de Laravel.
- **Piensa como atacante:** Si tu ataque no funciona, verifica que realmente estés usando código vulnerable (`echo` sin `escape`, `md5()` sin `salt`, consultas concatenadas).
- **Prueba en dos pestañas:** Para el ataque CSRF, abre `atacante.html` en una pestaña del mismo navegador donde tu app tiene sesión activa.

Responsabilidad Ética

Las técnicas aprendidas en esta tarea son exclusivamente para fines educativos y de investigación en seguridad. Aplicar estos conocimientos contra sistemas sin autorización explícita es ilegal y viola la ética profesional del ingeniero informático. El uso indebido de estas técnicas puede tener consecuencias legales graves.